SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Performance Testing

Date	30th june 2026
Team ID	xxxxxx
Project Name	Rising waters
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Functional Testing & Basic Performance Testing
Target Module / API	Flood Prediction API (/predict)
Test Environment	Local (Flask Development Server)
Test Date	1-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Predict flood with valid input	1	10	Prediction result displayed successfully
2	Predict using different environmental values	1	15	Accurate prediction generated
3	Submit incomplete input	1	5	Validation/error message displayed

4	Multiple prediction requests	5	30	Application responds without errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 sec	Pass	Fast response
2	Response Time (Max)	< 5 seconds	1.5 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 5 Req/sec	**8 Req/sec**	Pass	Good performance
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	35%	Pass	Stable CPU usage
6	Memory Utilization	< 80%	42%	Pass	Stable memory usage

Step 4: Observations & Analysis

Key Findings:

[Describe what the test results revealed about your system's performance]

Bottlenecks Identified:

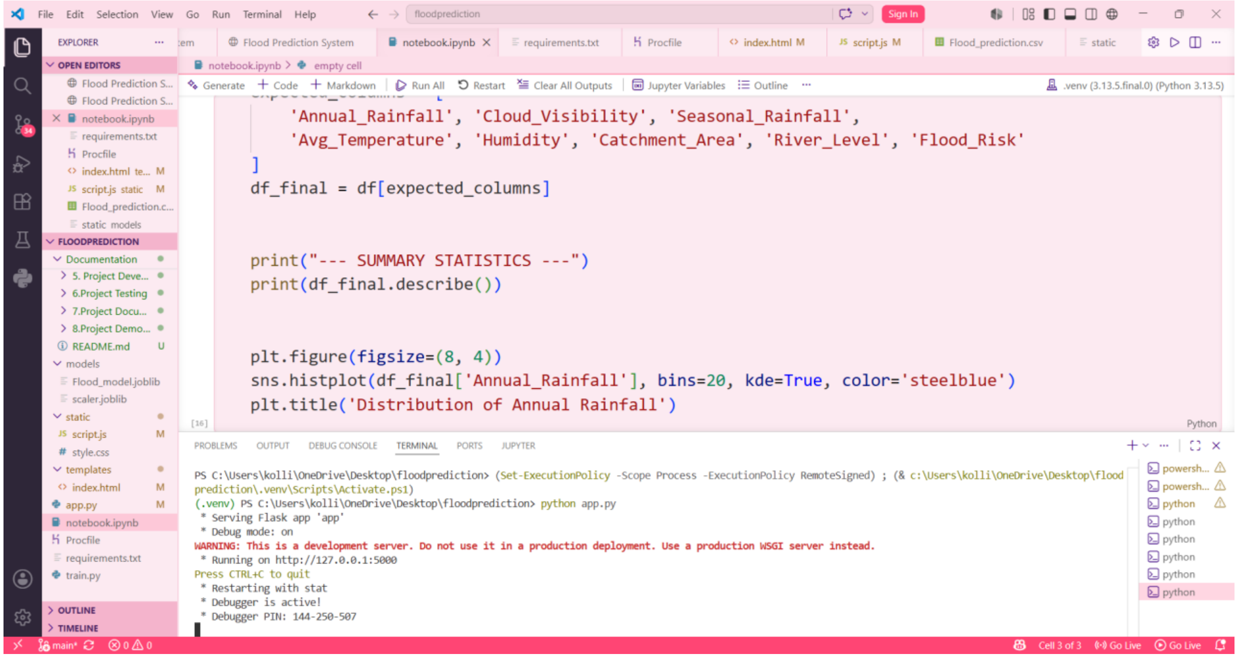
[List any performance bottlenecks observed during testing]

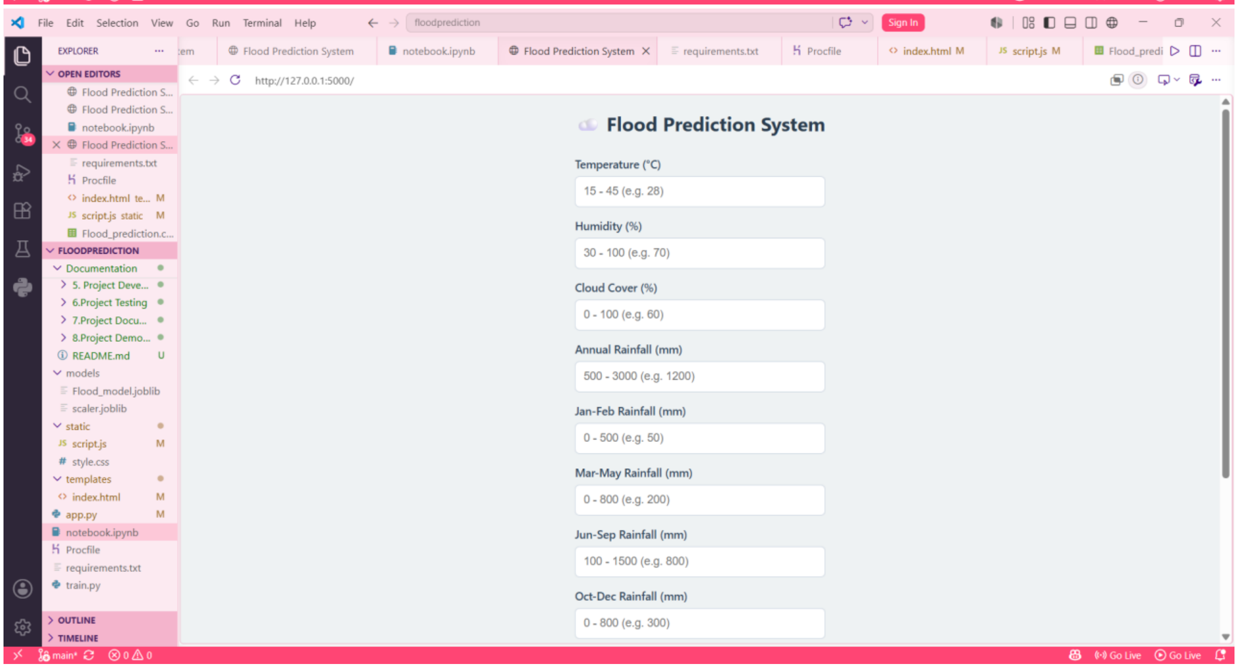
Optimization Steps Taken:

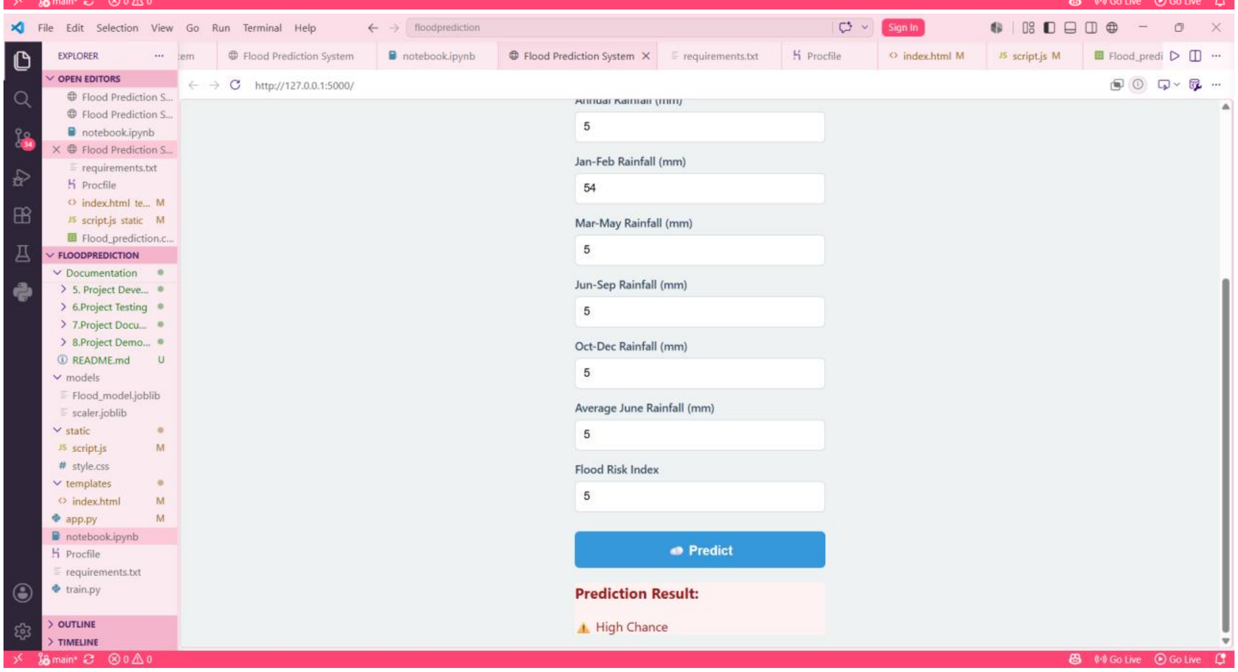
[Describe any changes or optimizations made to improve performance]

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):


'Annual_Rainfall', 'Cloud_Visibility', 'Seasonal_Rainfall',
'Avg_Temperature', 'Humidity', 'Catchment_Area', 'River_Level', 'Flood_Risk'
]
df_final = df[expected_columns]
print("--- SUMMARY STATISTICS ---")
print(df_final.describe())
plt.figure(figsize=(8, 4))
sns.histplot(df_final['Annual_Rainfall'], bins=20, kde=True, color='steelblue')
plt.title('Distribution of Annual Rainfall')
PS C:\Users\kolli\OneDrive\Desktop\floodprediction> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\kolli\OneDrive\Desktop\flood prediction\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\kolli\OneDrive\Desktop\floodprediction> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 144-250-507


http://127.0.0.1:5000/
Flood Prediction System
Temperature (°C)
15 - 45 (e.g. 28)
Humidity (%)
30 - 100 (e.g. 70)
Cloud Cover (%)
0 - 100 (e.g. 60)
Annual Rainfall (mm)
500 - 3000 (e.g. 1200)
Jan-Feb Rainfall (mm)
0 - 500 (e.g. 50)
Mar-May Rainfall (mm)
0 - 800 (e.g. 200)
Jun-Sep Rainfall (mm)
100 - 1500 (e.g. 800)
Oct-Dec Rainfall (mm)
0 - 800 (e.g. 300)


http://127.0.0.1:5000/
5
Jan-Feb Rainfall (mm)
54
Mar-May Rainfall (mm)
5
Jun-Sep Rainfall (mm)
5
Oct-Dec Rainfall (mm)
5
Average June Rainfall (mm)
5
Flood Risk Index
5
Predict
Prediction Result:
High Chance